# Computer Science Theory Reference

DCU International Foundation Year

Dr. Séamus Cummins

*Plain-English explanations, worked examples and revision tables*

Course: IFYCO002 Academic year: 2026–2027

---

### How to use this reference

Read the explanation, cover the worked answer, and try the example yourself. Learn what each term means, why it matters, and when you would use it. For an exam answer, connect the idea to the situation in the question rather than only repeating a definition.

This reference follows all nine learning outcomes in the NCUK *IFY Computer Science Module Specification v1.2*. It complements the Python cheat sheet and terminology glossary. It includes the theory behind practical work, but students should also practise writing programs, drawing diagrams, constructing databases and testing their solutions. A summary cannot replace those practical skills.

## Contents

**1 Computer systems and architecture (LO1) 2**

**2 Data representation (LO2) 6**

**3 Logical reasoning and algorithm representation (LO3) 10**

**4 Python, testing, algorithms and data structures (LO4) 13**

**5 Computational thinking and problem solving (LO5) 22**

**6 Software, languages and development (LO6) 24**

**7 Relational databases and SQL (LO7) 27**

**8 Networks and communication (LO8) 31**

**9 Cyber security, legislation, ethics and AI (LO9) 34**

**10 Revision checks and worked answers 39**

**Sources and further reading 41**

# 1 Computer systems and architecture (LO1)

## 1.1 The main components

A computer takes **input**, follows instructions to **process** data, produces **output**, and may **store** the results. Hardware is the physical equipment. Software is the programs and associated instructions that tell the hardware what to do.

| Term or idea | Meaning, use and example |
|---|---|
| **CPU** | The central processing unit executes instructions. It includes the control unit, arithmetic logic unit and registers. |
| **ALU** | The arithmetic logic unit performs arithmetic and logical operations, such as adding numbers and comparing values. |
| **Control unit (CU)** | Decodes instructions and coordinates operations using control signals. It directs data movement and execution. |
| **Register** | A very small, fast storage location inside the CPU. Registers hold values needed immediately, such as an address or instruction. |
| **RAM** | Working memory holding programs and data currently in use. It is normally volatile: its contents are lost when power is removed. |
| **Secondary storage** | Long-term storage, such as an SSD or HDD. It retains files when power is off, but access is normally slower than RAM. |
| **Input device** | Supplies data to the computer, for example a keyboard, microphone, camera or sensor. |
| **Output device** | Presents or acts on processed data, for example a display, speaker, printer or actuator. A touchscreen provides both input and output. |
| **Embedded system** | A computer built into a larger device to perform specific tasks, such as controlling a washing machine. It may have limited power, memory and processing resources. |

## 1.2 Registers, buses and the instruction cycle

| Term or idea | Meaning, use and example |
|---|---|
| **PC** | The program counter holds the address of the next instruction to fetch. A branch instruction may change it. |
| **MAR** | The memory address register holds the address of the memory location being accessed. An address identifies a location, not its contents. |
| **MDR** | The memory data register holds data or an instruction being transferred to or from memory. It is sometimes called the memory buffer register. |
| **CIR / IR** | The current instruction register holds the instruction being decoded and executed. |
| **ACC** | The accumulator holds intermediate arithmetic or logic results in architectures that use it. Real processors may use many general-purpose registers. |
| **Address bus** | Carries the address of a location to access. In a simple CPU–memory diagram it runs from the CPU to memory. More address bits allow more locations to be identified. |

| Term or idea | Meaning, use and example |
|---|---|
| **Data bus** | Carries data and instructions. It is normally bidirectional because the CPU both reads and writes data. |
| **Control bus** | Carries control and timing signals, including read, write and interrupt signals. Different signals travel in different directions. |

**Worked example: Fetch–decode–execute**

A simplified model is:

1. **Fetch:** copy the PC address into the MAR. Send a memory-read signal. Memory returns the instruction through the data bus into the MDR, then the instruction moves to the CIR. Update the PC to the next instruction address.
2. **Decode:** the control unit interprets the operation and identifies any operands needed.
3. **Execute:** carry out the operation, which might calculate, compare, access memory or change the PC. Store the result where required.
4. Repeat, unless execution stops. A branch may replace the next sequential PC value.

**Remember.** The MAR stores an address. The MDR stores the value being transferred. The PC stores the next instruction address, not the next instruction itself.

## 1.3 System architectures and data flow

| Term or idea | Meaning, use and example |
|---|---|
| **Von Neumann** | Instructions and data share memory and a communication path. The shared path can limit throughput when instruction and data transfers compete. |
| **Harvard** | Instructions and data have separate memories and paths, allowing simultaneous access. This adds hardware complexity and can suit embedded processing. |
| **Hybrid architecture** | Combines ideas, for example separate instruction and data caches with shared main memory. Many modern designs do not fit one simple textbook model exactly. |
| **Architecture diagram** | Shows components and connections. Follow addresses, values and control signals separately. Identify which memory holds instructions and which holds data. |
| **Data-flow diagram (DFD)** | Shows how information moves through a system, rather than the order in which program instructions execute. A process transforms data, a store retains it, an external entity supplies or receives it, and labelled arrows show flows. |

**Worked example: DFD for a library loan**

An external entity, *student*, supplies a book request to the *issue loan* process. The process reads book availability from the *books* data store, writes a loan record to the *loans* store and returns a receipt to the student. Label flows with information, such as “book ID”, rather than actions such as “click”.

## 1.4 Memory and storage

| Term or idea | Meaning, use and example |
|---|---|
| **ROM / firmware** | Non-volatile memory stores instructions such as startup firmware. Modern firmware is often held in rewritable flash memory rather than permanently fixed ROM. |
| **Cache** | Small, fast memory close to the CPU holding copies of useful instructions and data. A cache hit finds the item there; a miss requires access to a slower level. |
| **L1, L2 and L3** | Cache levels. L1 is usually smallest and fastest, L2 larger and slower, and L3 larger again and often shared between cores. Exact arrangements vary. |
| **Virtual memory** | An address-space mechanism that can use secondary storage to hold pages not currently in RAM. Accessing a page on storage is much slower than accessing RAM. Heavy paging can cause severe slowdown. |
| **Flash memory** | Non-volatile electronic memory used in SSDs, USB drives and memory cards. It has no moving mechanical parts. |
| **Magnetic storage** | An HDD stores data magnetically on rotating platters. It commonly offers large capacity at low cost per unit of storage, but mechanical movement adds delay and vulnerability to shocks. |
| **Optical storage** | CDs, DVDs and Blu-ray discs use a laser to read encoded information. Useful for distribution or some archives, but capacity and access speed are limited relative to many modern drives. |
| **Solid-state storage** | SSDs have low access latency and resist mechanical shocks. Cost, write endurance and the required capacity still matter when choosing a device. |
| **Cloud storage** | Files held on a provider's remote systems and accessed over a network. Offers remote access and managed capacity, but depends on connectivity, account security and provider arrangements. |
| **Virtual storage** | Presents logical storage that may combine or divide physical devices. It is different from virtual memory, which concerns a program's memory address space. |

Choose storage using **capacity, access speed, portability, durability, reliability and cost**. No medium removes the need for a backup. Synchronisation alone is not necessarily a backup because it may copy accidental deletion to every device.

## 1.5 Operating-system responsibilities

| Term or idea | Meaning, use and example |
|---|---|
| **Process management** | Starts, schedules, pauses and ends running programs. Context switching saves the state of one process and restores another. Switching has a cost. |
| **Memory management** | Tracks memory use and separates processes' memory. Paging uses fixed-size blocks; segmentation divides memory into logical, variable-size regions. |
| **File management** | Organises files and directories, records metadata and enforces access permissions. |
| **Device management** | Uses drivers to communicate with hardware. Buffers temporarily hold data when components operate at different rates. |

| Term or idea | Meaning, use and example |
|---|---|
| **Security management** | Manages user accounts, authentication, permissions and isolation between processes. |
| **Network management** | Configures connections and addressing and supports communication and resource sharing. |
| **Error handling** | Detects and records problems, reports failures and may attempt recovery. Logs support diagnosis. |

| Term or idea | Meaning, use and example |
|---|---|
| **First Come First Served** | Processes run in arrival order. Simple, but a long task can hold up many short tasks. |
| **Round Robin** | Each ready process receives a time slice in turn. Supports responsiveness, but very short slices increase context-switch overhead. |
| **Shortest Job First** | Selects the shortest estimated next job. Can reduce average waiting time, but long jobs may wait excessively and job length must be estimated. |
| **Shortest Remaining Time** | Pre-emptive version: run the job with the least estimated time left, switching if a shorter one becomes ready. |
| **Multi-Level Feedback Queue** | Uses queues with different priorities and time slices. Moves jobs between queues according to behaviour, aiming to support interactive tasks while preventing starvation through suitable policies. |

| Term or idea | Meaning, use and example |
|---|---|
| **Real-time OS** | Designed to meet timing deadlines, such as responding to a control sensor. "Real-time" means predictable timing, not simply fast performance. |
| **Embedded OS** | Designed for a dedicated device, often with constrained resources. An embedded system may also be real-time. |
| **Multitasking OS** | Supports progress on multiple tasks through scheduling, potentially also using several cores. |
| **Multi-user OS** | Allows multiple users to use resources while separating their work and permissions. |
| **Distributed OS** | Coordinates resources across connected computers and can present them as an integrated system. |

## 1.6 Performance and bottlenecks

| Term or idea | Meaning, use and example |
|---|---|
| **Clock speed** | Cycles per second, often measured in GHz. A higher rate can help, but different CPUs can complete different amounts of work per cycle. |
| **Cores** | Multiple processing units can execute work concurrently. Benefits depend on how much of the workload can run in parallel. Doubling cores does not always halve runtime. |
| **Pipelining** | Overlaps stages of different instructions. It can improve throughput, although dependencies and branches may cause stalls. It does not necessarily reduce one instruction's latency. |

| Term or idea | Meaning, use and example |
|---|---|
| **RAM capacity and speed** | Enough RAM reduces paging. Bandwidth measures data transferable per second; latency measures delay before an access completes. |
| **GPU** | Performs many similar operations in parallel, useful for graphics and suitable numerical tasks. Integrated GPUs share system resources; dedicated GPUs typically have their own memory. |
| **Bus width and speed** | A wider bus can carry more bits per transfer. Transfer rate also depends on clocking and the rest of the system. |
| **Cooling** | Excess heat can cause thermal throttling, where a component reduces speed to control temperature. |
| **Maintenance** | Unneeded software, malware and poor resource management can reduce performance. HDD fragmentation can add seek overhead; traditional defragmentation is not a general SSD speed remedy. |
| **Network limitation** | A fast CPU cannot remove a slow remote connection. Bandwidth, latency and server performance affect online tasks. |

**Worked example: Choosing an upgrade**

A laptop repeatedly moves memory pages to its SSD when many applications are open. More RAM may help more than a faster CPU because memory capacity is the current bottleneck. If the task is downloading a large remote file over a slow connection, extra RAM may make little difference.

**Check your understanding.** Why does increasing clock speed not guarantee that one computer will outperform another? Give two other relevant factors.

# 2 Data representation (LO2)

## 2.1 Bits, bytes and number systems

A **bit** is a binary digit, 0 or 1. A **byte** contains 8 bits. The same bit pattern can represent a number, text, part of a picture or an instruction; its meaning depends on the encoding and context.

| Term or idea | Meaning, use and example |
|---|---|
| **Decimal (base 10)** | Uses digits 0–9. Positions have weights $1, 10, 100, \ldots$ from right to left. |
| **Binary (base 2)** | Uses 0 and 1. Positions have weights $1, 2, 4, 8, 16, 32, 64, 128, \ldots$. |
| **Hexadecimal (base 16)** | Uses 0–9 and A–F, where A=10 through F=15. One hex digit represents exactly four binary bits. |
| **Unsigned integer** | With $n$ bits, represents 0 to $2^n - 1$. Eight bits give 256 possible values, 0–255. |
| **Decimal storage units** | 1 kB = 1000 bytes and 1 MB = $10^6$ bytes. |
| **Binary storage units** | 1 KiB = 1024 bytes and 1 MiB = $2^{20}$ bytes. State the convention if an exam uses KB ambiguously. |

**Worked example: Converting 45**

$$45_{10} = 32 + 8 + 4 + 1 = 00101101_2 = 2D_{16}.$$

For binary to decimal, add the weights with a 1. For decimal to binary, select descending powers of two or repeatedly divide by 2 and read the remainders upwards. For binary to hex, group bits in fours from the right: 0010 1101 gives 2D. Leading zeros can complete the first group without changing an unsigned value.

Hex is a compact way to display binary, commonly in addresses and colour codes. In RGB colour `#FF8000`, the red, green and blue channels are 255, 128 and 0.

## 2.2 Negative numbers and arithmetic

| Term or idea | Meaning, use and example |
|---|---|
| **Sign and magnitude** | The leading bit gives the sign, usually 0 for positive and 1 for negative. Remaining bits give magnitude. Eight bits represent $-127$ to $+127$ and include two encodings of zero. |
| **Two's complement** | The leading bit has weight $-2^{n-1}$; remaining weights are positive. Range: $-2^{n-1}$ to $2^{n-1} - 1$. Eight bits represent $-128$ to 127, with one zero. |
| **Negating in two's complement** | At a fixed bit width, invert all bits and add 1. The most negative value cannot be represented as a positive value at the same width. |
| **Overflow** | A result lies outside the range of the chosen representation. In unsigned addition, a carry beyond the width signals overflow. In signed two's-complement addition, adding same-sign inputs and obtaining the opposite sign signals overflow. |
| **Precision error** | Many fractions, such as decimal 0.1, have no finite binary expansion. Floating-point values store approximations, which can accumulate small errors. |
| **Rounding error** | Discarding detail to fit a limited precision changes a value slightly. Round at an appropriate stage, especially when displaying results. |

**Worked example: Representing $-13$ in eight bits**

$+13 = 00001101$. Sign and magnitude gives 10001101. Two's complement gives $11110010 + 1 = 11110011$. Check: $-128 + 64 + 32 + 16 + 2 + 1 = -13$.

**Worked example: Binary addition and overflow**

$0101 + 0011 = 1000$ in four bits. As *unsigned* values, $5 + 3 = 8$ fits. As *signed two's-complement* values, the range is $-8$ to 7, so the same calculation overflows and the result pattern represents $-8$. Always identify the representation first.

For subtraction, add the two's complement of the subtrahend. In eight bits, $7 - 3$ is $00000111 + 11111101 = 1\,00000100$. Keep the eight-bit result $00000100 = 4$. A discarded carry is not by itself signed overflow.

## 2.3 Text

| Term or idea | Meaning, use and example |
|---|---|
| **Character set** | A collection of characters and their assigned codes. |
| **ASCII** | The original 7-bit code has 128 values, including Latin letters, digits, punctuation and control characters. For example, A is decimal 65. "Extended ASCII" refers to differing 8-bit schemes, not one universal standard. |
| **Unicode** | Assigns code points across many writing systems and symbols, enabling multilingual text. It is not limited to one byte per character. |
| **UTF-8** | An encoding of Unicode code points using one to four bytes. ASCII characters use one byte. Character count and byte count can differ. |

**Worked example: Text size**

A file containing 1000 one-byte ASCII characters uses 1000 bytes for those characters, before any additional metadata. You cannot assume the same size for 1000 arbitrary Unicode characters without knowing the encoding and the characters.

## 2.4 Images

A bitmap is a grid of **pixels**. Its dimensions give width and height in pixels. **Colour depth** is the number of bits used per pixel, giving at most $2^b$ different pixel values at depth $b$.

$$\text{raw image size in bytes} = \frac{\text{width} \times \text{height} \times \text{bits per pixel}}{8}.$$

**Worked example: An 800 by 600 image at 24 bits per pixel**

$800 \times 600 \times 24/8 = 1\,440\,000$ bytes, or 1.44 MB. This is the uncompressed pixel data only. Headers, metadata, palettes, padding and compression can change the actual file size.

| Term or idea | Meaning, use and example |
|---|---|
| **Higher dimensions** | More pixels can preserve more spatial detail, but increase storage. Enlarging a small image does not recreate detail that was never captured. |
| **Higher colour depth** | Allows more colours or levels and smoother gradients, but increases raw size. Standard 24-bit RGB uses 8 bits for each colour channel. |
| **JPEG** | Commonly uses lossy compression, useful for photographs. Compression artefacts can appear around sharp edges or at high compression. |
| **PNG** | Uses lossless compression and supports transparency. Often suitable for diagrams, screenshots and images requiring exact pixel preservation. |

## 2.5 Sound

A microphone produces a changing analogue signal. Digitisation samples it at regular times and quantises each sample to one of a finite number of amplitude levels.

| Term or idea | Meaning, use and example |
|---|---|
| **Sample rate** | Samples per second per channel, measured in Hz. A higher rate can represent higher frequencies. For ideal band-limited sampling, the sample rate must exceed twice the highest frequency to be represented. |
| **Bit depth** | Bits per sample. More bits allow finer amplitude distinctions and less quantisation error. It does not mean more samples per second. |
| **Channels** | Independent audio streams, for example one for mono and two for stereo. |
| **WAV** | An audio container commonly holding uncompressed PCM sound, although other encodings can also be stored. |
| **MP3** | A lossy audio format that reduces data using properties of human hearing. Some original information cannot be recovered. |

$$\text{raw sound size in bytes} = \frac{\text{sample rate} \times \text{bit depth} \times \text{channels} \times \text{seconds}}{8}.$$

**Worked example: One minute of stereo PCM sound**

At 44,100 samples/s and 16 bits/sample: $44\,100 \times 16 \times 2 \times 60/8 = 10\,584\,000$ bytes, or 10.584 MB, excluding headers. Its raw bit rate is $1\,411\,200$ bits/s.

## 2.6 Compression

| Term or idea | Meaning, use and example |
|---|---|
| **Lossless** | Reconstructs the original data exactly. Appropriate for program files, text and records where losing a symbol would change meaning. |
| **Lossy** | Discards some information to obtain a smaller representation. Useful for some media, but the discarded detail cannot be restored exactly. |
| **Run-length encoding (RLE)** | Stores repeated symbols as a value and a count. For example, AAAAAABB becomes (A,6), (B,2). Good for long runs, but can make data larger when repetitions are short. |
| **Compression ratio** | Often defined as original size divided by compressed size. A 12 MB file reduced to 3 MB has a 4:1 ratio. State the convention used. |

**Remember.** Compression reduces the representation size. Encryption conceals the meaning. They solve different problems.

**Check your understanding.** Calculate the raw size of a 640 by 480 image at 8 bits per pixel. Why might its saved PNG file have a different size?

# 3 Logical reasoning and algorithm representation (LO3)

## 3.1 Analysing a problem

Before writing code, identify the **inputs**, required **outputs**, **processing**, data to retain and **constraints**. A constraint is a restriction, such as a maximum response time or a permitted score range. Requirements should be testable: "reject scores outside 0–100" is clearer than "handle scores properly".

**Decomposition** divides the problem into manageable parts. **Abstraction** removes irrelevant detail while retaining what matters. **Pattern recognition** identifies similarities that allow reuse. For a booking program, separate checking availability, recording the booking and producing confirmation. Each part needs an agreed input and output.

## 3.2 Boolean logic

A Boolean value is true or false, represented in logic by 1 or 0. Define what 1 means for each input before modelling a situation.

| **Operation** | **Expression** | **Output is true when** |
|---|---|---|
| NOT | $\neg A$ | A is false. |
| AND | $A \wedge B$ | Both inputs are true. |
| OR | $A \vee B$ | At least one input is true, including when both are true. |
| NAND | $\neg(A \wedge B)$ | The inputs are not both true. |
| NOR | $\neg(A \vee B)$ | Both inputs are false. |
| XOR | $A \oplus B$ | Exactly one of the two inputs is true. |
| XNOR | $\neg(A \oplus B)$ | The two inputs have the same value. |

| A | B | AND | OR | NAND | NOR | XOR | XNOR |
|---|---|---|---|---|---|---|---|
| 0 | 0 | 0 | 0 | 1 | 1 | 0 | 1 |
| 0 | 1 | 0 | 1 | 1 | 0 | 1 | 0 |
| 1 | 0 | 0 | 1 | 1 | 0 | 1 | 0 |
| 1 | 1 | 1 | 1 | 0 | 0 | 0 | 1 |

| A | NOT A |
|---|---|
| 0 | 1 |
| 1 | 0 |

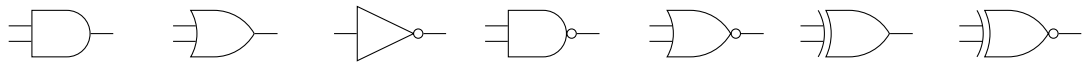
The usual precedence is brackets, NOT, AND, OR. Use explicit brackets with mixed operators, especially XOR. Alternative notation uses $AB$ for AND, $A + B$ for OR and $A'$ for NOT. Boolean + is not ordinary arithmetic: $1 \vee 1 = 1$.

## 3.3 Truth tables and circuits

A truth table with $n$ independent Boolean inputs has $2^n$ rows. For three inputs use 000, 001, 010, 011, 100, 101, 110, 111. Add intermediate columns rather than calculating a complicated expression mentally.

**Worked example:** $Q = (A \wedge \neg B) \vee C$

| A | B | C | $\neg B$ | $A \wedge \neg B$ | Q |
|---|---|---|---|---|---|
| 0 | 0 | 0 | 1 | 0 | 0 |
| 0 | 0 | 1 | 1 | 0 | 1 |
| 0 | 1 | 0 | 0 | 0 | 0 |
| 0 | 1 | 1 | 0 | 0 | 1 |
| 1 | 0 | 0 | 1 | 1 | 1 |
| 1 | 0 | 1 | 1 | 1 | 1 |
| 1 | 1 | 0 | 0 | 0 | 0 |
| 1 | 1 | 1 | 0 | 0 | 1 |




To build the circuit, pass B through NOT, combine that result with A using AND, then combine the result with C using OR. To read a circuit, label each intermediate wire with the subexpression it carries. A dot marks a connected wire junction; crossing lines without a dot are normally unconnected.

| **Term or idea** | **Meaning, use and example** |
|---|---|
| **Gate symbols** | AND has a flat input side and rounded output side. OR has a curved input side and pointed output. NOT is a triangle with an output circle. NAND and NOR add an output circle to AND and OR. XOR adds an extra curved line to OR. XNOR adds an output circle to XOR. |
| **Inversion bubble** | A small circle means NOT at that point. Its position determines which signal is inverted. |
| **Equivalent expressions** | Produce identical outputs for every input combination. One differing row disproves equivalence. |

### 3.4 Boolean algebra

| **Law** | **Forms to recognise** |
|---|---|
| Identity | $A \wedge 1 = A$; $A \vee 0 = A$ |
| Null | $A \wedge 0 = 0$; $A \vee 1 = 1$ |
| Complement | $A \wedge \neg A = 0$; $A \vee \neg A = 1$ |
| Idempotent | $A \wedge A = A$; $A \vee A = A$ |
| Double negation | $\neg(\neg A) = A$ |
| Commutative | $A \wedge B = B \wedge A$; $A \vee B = B \vee A$ |
| Associative | $(A \wedge B) \wedge C = A \wedge (B \wedge C)$; likewise for OR. |
| Distributive | $A \wedge (B \vee C) = (A \wedge B) \vee (A \wedge C)$ |
| Distributive (dual) | $A \vee (B \wedge C) = (A \vee B) \wedge (A \vee C)$ |
| Absorption | $A \vee (A \wedge B) = A$; $A \wedge (A \vee B) = A$ |
| De Morgan | $\neg(A \wedge B) = \neg A \vee \neg B$ |
| De Morgan | $\neg(A \vee B) = \neg A \wedge \neg B$ |

**Worked example: Simplifying with reasons**

$$\begin{aligned}(A \wedge B) \vee (A \wedge \neg B) &= A \wedge (B \vee \neg B) && \text{distributive}\\ &= A \wedge 1 && \text{complement}\\ &= A && \text{identity.}\end{aligned}$$

The output does not depend on B. For De Morgan's laws, negate every operand and swap AND with OR: $\neg(A \vee \neg B) = \neg A \wedge B$.

## 3.5 Karnaugh maps, up to four variables

A K-map rearranges truth-table outputs so neighbouring cells differ in one input bit. For three variables use rows A=0,1 and columns BC=00,01,11,10. For four variables use AB and CD, both ordered 00,01,11,10. This is **Gray-code order**, not ordinary binary counting.

1. Copy each truth-table output into the cell with the matching input labels.
2. Form rectangular groups of 1, 2, 4, 8 or 16 ones. Do not include a zero.
3. Use large useful groups and cover every 1. Groups may overlap. Opposite edges are adjacent, including all four corners. Diagonal adjacency does not count.
4. For each group, retain only inputs that stay constant. A constant 1 gives the unnegated input; a constant 0 gives its negation. Inputs that change disappear.
5. AND retained inputs within each term, then OR the terms together. Remove redundant groups where possible.

**Worked example: A three-variable map**

| A / BC | 00 | 01 | 11 | 10 |
|---|---|---|---|---|
| 0 | 0 | 0 | 1 | 0 |
| 1 | 1 | 1 | 1 | 0 |

The pair at A=1, BC=00 and 01 gives $A \wedge \neg B$. The pair in column BC=11 gives $B \wedge C$. Therefore $Q = (A \wedge \neg B) \vee (B \wedge C)$. A pair along the bottom row at BC=01 and 11 would give $A \wedge C$, but it covers no otherwise uncovered 1, so that extra term is unnecessary.

**Worked example: A four-variable wraparound group**

| AB / CD | 00 | 01 | 11 | 10 |
|---|---|---|---|---|
| 00 | 1 | 0 | 0 | 1 |
| 01 | 0 | 0 | 0 | 0 |
| 11 | 0 | 0 | 0 | 0 |
| 10 | 1 | 0 | 0 | 1 |

The four corners form one group. A and C change, but B=0 and D=0 throughout. The answer is $\neg B \wedge \neg D$. An all-zero map gives 0; an all-one map gives 1. An isolated 1 retains every input in its term.

### 3.6 Flowcharts and pseudocode

| Term or idea | Meaning, use and example |
|---|---|
| **Terminator** | Oval or rounded terminal symbol for Start or End. |
| **Process** | Rectangle for an operation, for example calculating a total or assigning a variable. |
| **Input/output** | Parallelogram for receiving input or displaying output. |
| **Decision** | Diamond containing a condition. Label outgoing branches, usually True/False or Yes/No. |
| **Flow line** | Arrow showing the direction of control. A loop includes a route back to an earlier step. |
| **Subroutine** | Rectangle with additional vertical side lines for a named procedure or function defined elsewhere. |
| **Pseudocode** | A clear, language-independent description of an algorithm. Indentation and explicit block endings show structure. Follow the notation supplied in the question. |

**Worked example: Validation as pseudocode**

```
INPUT score
WHILE score < 0 OR score > 100
    OUTPUT "Enter a score from 0 to 100"
    INPUT score
ENDWHILE
IF score >= 50 THEN
    OUTPUT "Pass"
ELSE
    OUTPUT "Not yet passed"
ENDIF
```

In a flowchart, the validation diamond's invalid branch leads to a message, then another input, then back to the diamond. Its valid branch proceeds to the pass decision. A DFD shows data movement; a flowchart shows control flow.

**Check your understanding.** Simplify $X \vee (X \wedge Y)$. Name the law. Explain why a three-input truth table needs eight rows.

## 4 Python, testing, algorithms and data structures (LO4)

### 4.1 Programming essentials

Python is a high-level, general-purpose language used for automation, data analysis, web services and AI. An **IDE** combines tools such as an editor, syntax highlighting, execution console and debugger. A breakpoint pauses execution so you can inspect state. Step-through execution runs a small part at a time. Version control records changes and helps collaboration.

| Term or idea | Meaning, use and example |
|---|---|
| **Variable** | A name referring to a value. Assignment binds a name to a value, for example `score = 75`. |

| Term or idea | Meaning, use and example |
|---|---|
| **Constant** | A value intended not to change. Python names such as `MAX_SCORE` signal this convention, but ordinary uppercase names are not automatically enforced constants. |
| **Expression** | Code that produces a value, for example `price * quantity`. |
| **Statement** | An instruction, such as assignment, a loop or a return statement. |
| **Integer / int** | Whole number, such as `-12`. Ordinary Python integers can grow beyond a fixed machine word, subject to available memory. |
| **Float** | Approximate floating-point number, such as `3.5`. Some decimal fractions cannot be represented exactly. |
| **Boolean / bool** | `True` or `False`. Used for conditions. |
| **String / str** | An immutable sequence of characters, such as `"Dublin"`. Quotes distinguish text from a variable name. |
| **Type conversion** | `int("12")` gives 12; `float("2.5")` gives 2.5. Invalid text raises an error. `int(2.9)` truncates towards zero, giving 2. |
| **Input and output** | `input()` returns text. Convert it if a number is needed. `print()` displays values; displaying a value is not the same as returning it from a function. |
| **Indentation** | Defines code blocks. Use consistent indentation, conventionally four spaces, and a colon after headers such as `if`, `for` and `def`. |
| **Comment** | Text after `#` explains intent to a reader. Good comments explain why a non-obvious choice was made. |

### 4.2 Operators: one operation per row

| Operator | Meaning | Example | Result |
|---|---|---|---|
| `+` | Addition | `7 + 3` | `10` |
| `-` | Subtraction | `7 - 3` | `4` |
| `*` | Multiplication | `7 * 3` | `21` |
| `/` | True division | `7 / 2` | `3.5` |
| `//` | Floor division | `7 // 2` | `3` |
| `%` | Remainder | `7 % 2` | `1` |
| `**` | Exponentiation | `2 ** 3` | `8` |
| `==` | Equal values | `5 == 5` | `True` |
| `!=` | Unequal values | `5 != 5` | `False` |
| `<` | Less than | `3 < 5` | `True` |
| `<=` | Less than or equal | `5 <= 5` | `True` |
| `>` | Greater than | `3 > 5` | `False` |
| `>=` | Greater than or equal | `5 >= 5` | `True` |
| `and` | Both conditions true | `True and False` | `False` |
| `or` | At least one true | `True or False` | `True` |
| `not` | Logical negation | `not True` | `False` |
| `in` | Membership | `3 in [1, 3]` | `True` |
| `not in` | Non-membership | `2 not in [1, 3]` | `True` |

| Operator | Meaning | Example | Result |
|---|---|---|---|
| `=` | Assignment | `x = 5` | x becomes 5 |
| `+=` | Add and assign | `x += 2`, initially 5 | x becomes 7 |

**Remember.** Floor division rounds down: `-7 // 2` is -4. Python `**` means power; `^` is bitwise XOR. Assignment `=` and comparison `==` have different jobs.

Use brackets when grouping may be unclear. Arithmetic is evaluated before comparisons, then `not`, `and`, `or`. Exponentiation binds more tightly than a unary minus: `-2 ** 2` is -4, while `(-2) ** 2` is 4.

### 4.3 Strings and control flow

| Term or idea | Meaning, use and example |
|---|---|
| **String operations** | `len(text)` gives length; `text[0]` gives the first character; `text[-1]` gives the last. `text[1:4]` takes indices 1,2,3. The end index is excluded. |
| **String methods** | `strip()` removes surrounding whitespace; `lower()` changes case; `split()` creates parts; `replace()` substitutes text. Strings are immutable, so methods return new strings rather than editing the original. |
| **Formatting** | An f-string inserts values: `f"Total: {total:.2f}"` displays two decimal places. This controls presentation, not the exact internal value. |
| **Sequence** | Execute instructions in order. An assignment must happen before the value is used. |
| **Selection** | `if/elif/else` chooses a branch. In one chain, only the first true branch runs. Separate `if` statements are evaluated separately. |
| **For loop** | Iterates over a sequence or range. `range(2, 8, 2)` supplies 2,4,6. The stop value is excluded. |
| **While loop** | Repeats while its condition is true. It may run zero times. Something must eventually make the condition false unless repetition is intentionally indefinite. |
| **Nested loops** | One loop inside another. For each outer iteration, the inner loop completes its own iterations. Useful for 2D data. |
| **Break** | Ends the nearest enclosing loop immediately. |
| **Continue** | Skips the rest of the current iteration and proceeds to the next one. |

**Worked example: A running total and trace**

```
total = 0
for number in range(1, 4):
    total += number
print(total)
```

| Iteration | number | total after addition |
|---|---|---|
| 1 | 1 | 1 |
| 2 | 2 | 3 |
| 3 | 3 | 6 |

The output is 6. Initialising `total` inside the loop would erase the accumulated value each time.

**Worked example: A common Boolean bug**

```
# Wrong: the non-empty string "blue" is truthy.
if colour == "red" or "blue":
    print("Accepted")

# Correct: compare on both sides.
if colour == "red" or colour == "blue":
    print("Accepted")
```

Empty strings, empty collections, zero and `None` are false in a condition; most other values are true. `bool("False")` is true because the string is non-empty. For Boolean operands, `and/or` behave as in truth tables; with other operands they return an operand. They short-circuit: `and` stops at a false left operand and `or` at a true left operand.

## 4.4 Functions, scope and modularity

| Term or idea | Meaning, use and example |
|---|---|
| **Function** | A named reusable unit of code. Define it with `def`, then call it to run its body. |
| **Parameter** | A name in the function definition, such as `width` in `def area(width, height)`. |
| **Argument** | A value supplied in a call, such as 4 in `area(4, 3)`. |
| **Return** | Sends a result back to the caller and ends that function call. A Python function without an explicit return value returns `None`. |
| **Procedure** | A subroutine used mainly for an action, such as displaying output. Python uses `def` for both functions and procedures. |
| **Local scope** | A name defined inside a function normally belongs to that function call. Pass data through parameters and return values to make dependencies clear. |
| **Modularity** | Breaking a program into focused functions improves reuse, readability, independent testing and debugging. |

```
def rectangle_area(width, height):
    return width * height

area = rectangle_area(4, 3)
print(area)  # 12
```

A function that only prints 12 does not supply 12 to a calculation in the caller. Choose `return` when another part of the program needs the result.

## 4.5 Collections and dimensions

| Term or idea | Meaning, use and example |
|---|---|
| **List** | A mutable ordered collection. `scores = [40, 70, 90]`; `scores[1] = 75` changes the second item. Lists support mixed types, though consistent types often simplify processing. |
| **Array** | An indexed collection, often of one data type and a defined shape. The course allows Python lists for basic array work, but lists and specialised numeric arrays are not identical. |
| **Tuple** | An immutable sequence, useful for a fixed group such as coordinates `(3, 5)`. You cannot replace a tuple element, although an element may itself refer to a mutable object. A one-item tuple needs a comma: `(3,)`. |
| **Dictionary** | A mapping from unique keys to values, such as {`"name": "Amina", "score": 82`}. Retrieve by key rather than numeric position. Keys must be hashable, such as strings or integers. |
| **List methods** | `append(x)` adds at the end; `pop()` removes and returns the last item; `remove(x)` removes the first matching value. Most mutating methods, such as `append`, return `None`. |
| **Dictionary access** | `student["score"]` retrieves a key or raises `KeyError`. `student.get("score", 0)` supplies a default if missing. `items()` gives key–value pairs for iteration. |
| **2D data** | A grid accessed by row then column: `grid[row][column]`. A rectangular list with 3 rows of 4 elements has shape $3 \times 4$ and 12 elements. |
| **3D and nD data** | Additional dimensions represent more axes, for example time, row and column. Total elements in a rectangular array are the product of its dimension sizes. |

**Worked example: A 2D list**

```
grid = [[1, 2, 3], [4, 5, 6]]
print(grid[1][2])  # 6
for row in grid:
    print(sum(row))  # 6, then 15
```

Avoid `[[0] * 3] * 2` when rows must change independently: it repeats references to the same inner list. Use `[[0] * 3 for _ in range(2)]`.

## 4.6 Stacks, queues, linked lists and trees

| Term or idea | Meaning, use and example |
|---|---|
| **Stack** | Last In, First Out (LIFO). Push adds at the top; pop removes from the top; peek reads without removing. Useful for undo operations and function-call management. |
| **Queue** | First In, First Out (FIFO). Enqueue adds at the rear; dequeue removes from the front. Useful for service requests and print jobs. |
| **Singly linked list** | A sequence of nodes, each containing data and a reference to the next node. The head references the first node; the final next reference is null, represented by `None` in Python. |

| Term or idea | Meaning, use and example |
|---|---|
| **Linked-list traversal** | Start at the head, process the current node, then follow its next reference until null. Searching is normally linear. There is no constant-time arbitrary index access. |
| **Linked-list insertion** | To insert after node P, set the new node's next to P's old next, then set P's next to the new node. The links must be updated without losing the remaining list. |
| **Linked-list deletion** | To remove the node after P, redirect P's next to the removed node's next. Removing the head updates the head reference. Finding the correct position may take linear time. |
| **Binary tree** | A hierarchical structure in which each node has at most two children. Terms include root, parent, child, leaf and subtree. |
| **Binary search tree (BST)** | An ordered binary tree: values in the left subtree are smaller and values in the right subtree are larger, with a defined policy for duplicates. Not every binary tree is a BST. |

```
stack = []
stack.append("first")  # push
stack.append("second")
if stack:              # not empty
    print(stack[-1])   # peek: second
    print(stack.pop()) # pop: second

from collections import deque
queue = deque()
queue.append("first")  # enqueue
queue.append("second")
if queue:
    print(queue.popleft())  # dequeue: first
```

A list can model a queue with `pop(0)`, but this shifts later elements and takes $O(n)$ time. A deque supports efficient operations at both ends. Test for emptiness before popping if an empty structure is possible.

**Worked example: Tree traversal**

Consider a tree with root 4, left child 2 (children 1 and 3), and right child 6 (children 5 and 7).

| Term or idea | Meaning, use and example |
|---|---|
| **In-order** | Left subtree, node, right subtree: 1,2,3,4,5,6,7. Gives sorted order for a BST. |
| **Pre-order** | Node, left subtree, right subtree: 4,2,1,3,6,5,7. |
| **Post-order** | Left subtree, right subtree, node: 1,3,2,5,7,6,4. |

A balanced BST can search in $O(\log n)$ time. A badly skewed BST can behave like a list, taking $O(n)$. Lists suit indexed access, linked lists suit link changes when the relevant node is known, stacks suit reverse order, queues suit arrival order, and dictionaries suit key lookup.

## 4.7 Files, modules and turtle graphics

| Term or idea | Meaning, use and example |
|---|---|
| **File mode `r`** | Read an existing file. Opening a missing file raises an exception. |
| **File mode `w`** | Write a file, creating it if needed and truncating an existing file. |
| **File mode `a`** | Append at the end, creating the file if necessary. |
| **File operations** | `read()` reads contents; `write(text)` writes a string; `close()` releases the resource. A `with` block closes the file automatically. |
| **Module / library** | Reusable code imported into a program. `math.sqrt(9)` gives 3.0; `random.randint(1, 6)` includes both endpoints. |
| **Turtle state** | A turtle has a position, heading, pen state and drawing settings. `forward`, `left` and `right` change its path. `penup` allows movement without a line. |
| **Turtle coordinates** | Typically the screen centre is (0,0), positive x is right, positive y is up and the initial heading is right. `goto(x,y)` uses an absolute position. |

```
with open("scores.txt", "w", encoding="utf-8") as file:
    file.write("75\n82\n")

with open("scores.txt", "r", encoding="utf-8") as file:
    scores = [int(line.strip()) for line in file]
```

**Worked example: Iteration with turtle**

```
import turtle
pen = turtle.Turtle()
for side in range(6):
    pen.forward(60)
    pen.left(360 / 6)
turtle.done()
```

A regular polygon turns by its **exterior angle**, $360/n$, not its interior angle. Turtle normally needs a graphical environment.

## 4.8 Errors, testing and improvement

| Term or idea | Meaning, use and example |
|---|---|
| **Syntax error** | The source does not follow grammar rules, such as a missing colon. The relevant code cannot run until corrected. |
| **Runtime error** | Occurs during execution, such as invalid conversion, division by zero or an out-of-range list index. |
| **Logic error** | The program runs but produces the wrong behaviour, such as calculating the wrong average. |
| **Validation** | Checks whether input follows rules, such as type, range, length, presence or allowed values. It does not prove the data is factually correct. |
| **Verification** | Checks data has been copied or entered as intended, for example comparing two entries or checking against a source. |
| **Functional testing** | Checks the program does what the requirements specify. |

| Term or idea | Meaning, use and example |
|---|---|
| **Manual / automated** | Manual testing uses a person to execute checks. Automated testing uses code to repeat checks and compare actual with expected results. |
| **Test plan / log** | Record test ID, purpose, input, expected result, actual result and pass/fail. Write the expected result before running the test. |
| **Normal test data** | Typical valid data, such as 65 for a score in 0–100. |
| **Boundary test data** | Values on and just around limits: -1,0,1 and 99,100,101. Some boundary tests deliberately use invalid inputs. |
| **Invalid test data** | Data that must be rejected, such as text where a score is required. |
| **Trace table** | Records changes in variables and outputs during a dry run. Include condition results where useful. |
| **Refactoring** | Changes code structure while preserving intended behaviour. It can improve clarity without making execution faster. |
| **Optimisation** | Improves a chosen property, such as time or memory use. Measure the bottleneck, change it and retest correctness. |

```
try:
    score = int(input("Score from 0 to 100: "))
    if not 0 <= score <= 100:
        print("Score is outside the allowed range.")
except ValueError:
    print("Please enter a whole number.")
```

Handle errors you expect, with useful messages. Do not hide every exception indiscriminately. Gather user feedback by observation or testing, prioritise changes, and rerun existing tests after a fix to detect regressions.

### 4.9 Searching and algorithm efficiency

| Term or idea | Meaning, use and example |
|---|---|
| **Linear search** | Check each item until the target is found or the data ends. Works on unsorted data. Best case $O(1)$, average and worst case $O(n)$. |
| **Binary search** | On sorted, efficiently indexed data, compare the middle item and discard the half that cannot contain the target. Best case $O(1)$; average and worst case $O(\log n)$. |
| **Time complexity** | How the amount of work grows as input size $n$ grows. Big O describes a growth bound, not the exact number of seconds. |
| **Space complexity** | How memory requirements grow. Distinguish total storage from auxiliary storage used in addition to the input. |
| **Best, average, worst** | Different inputs of the same size can require different work. Average-case analysis requires assumptions about the input distribution. State which case you mean. |

```
def linear_search(items, target):
    for i in range(len(items)):
        if items[i] == target:
            return i
    return -1

def binary_search(items, target):  # items must be sorted
    low, high = 0, len(items) - 1
    while low <= high:
        middle = (low + high) // 2
        if items[middle] == target:
            return middle
        if items[middle] < target:
            low = middle + 1
        else:
            high = middle - 1
    return -1
```

**Worked example: Binary-search trace**

Search for 12 in [2,5,8,12,16,23,38]. First low=0, high=6, middle=3, value=12: found. For 23, first discard indices 0–3, then low=4, high=6, middle=5, value=23: found. The updates use middle+1 or middle-1 because the middle item was already checked.

**Remember.** Sorting has a cost. For a single search in unsorted data, linear search may be better than sorting first. Binary search is especially useful for repeated searches of already sorted data.

## 4.10 Sorting

| Term or idea | Meaning, use and example |
|---|---|
| **Bubble sort** | Compare adjacent items and swap them if out of order. After one full left-to-right pass, the largest remaining item is at the right. Repeat over the unsorted portion. |
| **Merge sort** | Divide the data into halves until pieces contain at most one item, then merge sorted pieces by repeatedly choosing the smaller front item. |
| **Recursion** | A function calls itself on a smaller problem. A base case stops further calls. Merge sort uses recursion to express divide and conquer. |

```
def bubble_sort(items):
    for end in range(len(items) - 1, 0, -1):
        swapped = False
        for i in range(end):
            if items[i] > items[i + 1]:
                items[i], items[i + 1] = items[i + 1], items[i]
                swapped = True
        if not swapped:
            break
```

For [5,1,4,2], the first pass gives [1,4,2,5], the second [1,2,4,5]. This version modifies the list and does not return a new one.

```
def merge_sort(items):
    if len(items) <= 1:
        return items[:]
    mid = len(items) // 2
    left = merge_sort(items[:mid])
    right = merge_sort(items[mid:])
    result = []
    i = j = 0
    while i < len(left) and j < len(right):
        if left[i] <= right[j]:
            result.append(left[i])
            i += 1
        else:
            result.append(right[j])
            j += 1
    result.extend(left[i:])
    result.extend(right[j:])
    return result
```

For [5,1,4,2], split into [5,1] and [4,2], then singletons. Merge to [1,5] and [2,4], then [1,2,4,5]. The function returns a new sorted list.

| **Algorithm** | **Best time** | **Worst time** | **Auxiliary space** |
|---|---|---|---|
| Linear search | $O(1)$ | $O(n)$ | $O(1)$ |
| Iterative binary search | $O(1)$ | $O(\log n)$ | $O(1)$ |
| Bubble sort above | $O(n)$ | $O(n^2)$ | $O(1)$ |
| Merge sort above | $O(n \log n)$ | $O(n \log n)$ | $O(n)$ peak, plus recursion bookkeeping |

Bubble sort's average time is $O(n^2)$. Merge sort's average is $O(n \log n)$. Bubble sort is simple and uses little additional memory; merge sort scales better on large data but requires temporary storage in this implementation. A loop nested inside another is not automatically $O(n^2)$: count how many iterations actually occur.

**Check your understanding.** Choose a data structure for undo, a printer waiting list and student records looked up by ID. Why is binary search unsafe on unsorted data?

# 5 Computational thinking and problem solving (LO5)

Computational thinking means organising a problem so that a precise, repeatable solution can be developed. It is useful even before choosing a programming language.

| **Term or idea** | **Meaning, use and example** |
|---|---|
| **Decomposition** | Split a larger task into smaller parts with clear responsibilities. In a quiz: ask questions, check responses, calculate scores and display feedback. |
| **Pattern recognition** | Notice repeated structures or similarities. Many questions use the same ask–check–score process, suggesting a reusable function. |
| **Abstraction** | Retain essential features and omit irrelevant detail. A route planner needs locations and travel costs, not the colour of every building. |

| Term or idea | Meaning, use and example |
|---|---|
| **Algorithm design** | Specify an ordered, unambiguous procedure that transforms inputs into the required outputs and terminates for its intended valid inputs. |

## 5.1 Forms of abstraction

| Term or idea | Meaning, use and example |
|---|---|
| **Data abstraction** | Use a meaningful data model without exposing its internal representation. A stack offers push and pop regardless of whether a list or linked structure implements it. |
| **Procedural abstraction** | Give a sequence of operations a name and interface. A caller uses `calculate_average(scores)` without repeating its internal steps. |
| **Control abstraction** | Use constructs such as loops to express a control pattern without specifying every repeated instruction separately. |
| **Problem abstraction** | Represent the problem using only the details needed for the solution, such as modelling roads as connections between locations. |
| **Abstraction by specification** | Describe what an operation must do through its inputs, outputs and conditions, while leaving how it achieves this hidden. For example, a search returns a matching index or -1. |

## 5.2 Structured approaches

| Term or idea | Meaning, use and example |
|---|---|
| **Top-down design** | Begin with the complete goal, then divide it into increasingly detailed subproblems. |
| **Stepwise refinement** | Replace broad steps with more precise instructions until each step can be implemented. “Check input” becomes “convert to integer, test 0–100, report an error if invalid”. |
| **Divide and conquer** | Split into smaller instances of a problem, solve them and combine their results. Merge sort is an example. Not every division into program modules is a divide-and-conquer algorithm. |

**Worked example: Designing a class-average program**

1. **Define:** calculate the mean of valid student scores.
2. **Analyse:** input is a collection of scores from 0 to 100; output is the mean or an empty-data message. Reuse a validation rule.
3. **Design:** validate values, add them, count them, and divide total by count only when count is non-zero.
4. **Implement:** use a function with a clear return value.
5. **Test:** try [50,70], one score, an empty list and invalid entries. Expected mean for [50,70] is 60.

6. **Evaluate and improve:** check clarity and behaviour against the requirement. Avoid repeatedly calculating the same total.

A **precondition** states what must hold before an operation, such as "the list contains valid numeric scores". A **postcondition** states what will hold afterwards, such as "the returned value equals the arithmetic mean". State how empty data is handled rather than leaving it ambiguous.

**Check your understanding.** Explain the difference between decomposition and abstraction using the class-average example.

# 6 Software, languages and development (LO6)

## 6.1 Software categories and licensing

| Term or idea | Meaning, use and example |
|---|---|
| **System software** | Supports the computer's operation, including the OS, drivers and utilities. |
| **Utility software** | Performs maintenance or support tasks, such as backup, compression, malware scanning or disk management. |
| **Device driver** | Provides the software interface needed for an OS to control a particular device. |
| **General-purpose application** | Supports common tasks, such as word processing, spreadsheets, browsing or email. |
| **Specialist application** | Designed for a particular domain, such as CAD, accounting or medical imaging. It is not necessarily custom-written for one organisation. |
| **Open-source software** | Source code is available under a licence allowing specified use, modification and redistribution. Benefits can include inspection and adaptability; support and maintenance arrangements must still be assessed. |
| **Proprietary software** | The owner controls use through licence terms, with source code usually unavailable to users. It may offer commercial support, but can restrict modification and create dependence on a supplier. |

Open source does not necessarily mean zero cost. Proprietary does not automatically mean paid, secure or well supported. Compare total cost, support, interoperability, licensing obligations and long-term control.

## 6.2 Languages and translators

| Term or idea | Meaning, use and example |
|---|---|
| **Machine code** | Binary instructions executed by a processor. Instruction formats depend on the processor architecture. |
| **Assembly language** | Uses symbolic mnemonics and labels close to machine instructions. Provides hardware control but is less portable and more detailed than a high-level language. |

| Term or idea | Meaning, use and example |
|---|---|
| **High-level language** | Uses abstractions that make programs easier to express and maintain. Python, Java, JavaScript and C++ are examples, with different design and execution models. |
| **General-purpose language** | Can solve a broad range of tasks. Python is often used for automation and data work; Java and C++ are also used in large applications. |
| **Special-purpose language** | Focuses on a domain, such as SQL for database queries. It can express domain operations concisely. |
| **Assembler** | Translates assembly language into machine code. |
| **Compiler** | Translates source into another representation before its later execution, often machine code or bytecode. Compilation can provide optimisation and early error detection. |
| **Interpreter** | Executes a representation of a program through a runtime, which can support flexible experimentation but adds execution overhead compared with some compiled approaches. |
| **Hybrid execution** | Compilation and interpretation can coexist. CPython compiles source to bytecode and executes it in a virtual machine. Java commonly uses bytecode and just-in-time compilation. Languages are not inherently limited to one translation method. |

### 6.3 Programming paradigms

A paradigm is an approach to organising computation. A language can support several paradigms.

| Term or idea | Meaning, use and example |
|---|---|
| **Procedural** | Organises step-by-step instructions into procedures or functions. Straightforward for small tasks, but poorly separated shared state can make large systems difficult to maintain. |
| **Object-oriented** | Models objects containing state and behaviour. A class defines a kind of object; an instance is a particular object. Encapsulation controls access to internal state, and inheritance can specialise an existing class. Useful for related entities, but excessive class hierarchies add complexity. |
| **Functional** | Emphasises composition of functions and often immutability. A pure function returns the same result for the same inputs without observable side effects. This can aid testing, although real applications still need I/O and state management. |
| **Event-driven** | Responds to events, such as clicks, messages or sensor changes. Handlers run when events occur. Useful for interfaces, but the order of interactions can complicate reasoning. |
| **Logic programming** | Expresses facts and rules and asks queries; an inference system searches for solutions. Prolog is an example. Suitable for some rule-based problems, but execution may be less intuitive for beginners. |

## 6.4 The software development lifecycle

| Term or idea | Meaning, use and example |
|---|---|
| **Requirements** | Find out what users need and record measurable acceptance criteria. Include functional behaviour and constraints such as performance or accessibility. |
| **Analysis** | Understand the problem, existing processes, data, risks and feasibility. |
| **Design** | Plan algorithms, data structures, interfaces, database schemas and component responsibilities. |
| **Implementation** | Write and integrate the solution, using agreed coding practices. |
| **Testing** | Compare actual behaviour with requirements, including invalid inputs and boundaries. Record and repair defects. |
| **Deployment** | Make the solution available to its intended users and environment. Installation, configuration and data migration may be involved. |
| **Maintenance and evaluation** | Fix defects, adapt to new needs and assess whether the solution remains useful, efficient and usable. |
| **Documentation** | Records requirements, design decisions, interfaces, tests and usage instructions so work can be understood and maintained. |
| **Version control** | Tracks revisions, supports parallel work and allows changes to be reviewed or restored. It does not replace testing or project planning. |

## 6.5 Development approaches and evaluation

| Term or idea | Meaning, use and example |
|---|---|
| **Waterfall** | Uses planned sequential phases. Clear milestones suit stable requirements, but discovering a major misunderstanding late can make changes expensive. |
| **Agile** | Develops in short iterations with feedback and reprioritisation. Suits evolving requirements, but needs engaged stakeholders and disciplined management of scope and documentation. Agile does not mean no planning or documentation. |
| **RAD** | Rapid Application Development emphasises prototypes and quick user feedback. Can clarify interfaces early, but speed may compromise architecture if prototypes become production systems without refinement. |
| **Spiral** | Repeats cycles of objectives, risk analysis, development and review. Useful where risk is central, but requires expertise and adds management overhead. |
| **DevOps** | Integrates development and operations practices, including automated testing, deployment pipelines and monitoring. Can improve delivery reliability, but requires suitable automation and coordination. It can complement Agile rather than replace it. |

**Worked example: Choosing an approach**

An interface with unclear requirements benefits from prototyping and frequent feedback. A safety-sensitive system needs strong risk analysis, traceability and validation, whatever lifecycle labels are used. A small script may be clearest procedurally, while a larger interface may combine objects with event handlers. Choose based on requirements, team skills, scale, maintainability and risk, not popularity alone.

**Check your understanding.** Give one advantage and one limitation of Agile for a project whose requirements change frequently.

# 7 Relational databases and SQL (LO7)

## 7.1 Tables, entities and relationships

A database stores organised data. A DBMS, or database management system, provides mechanisms to store, retrieve, update and control access to it. A relational database uses related tables to avoid unnecessary duplication and maintain consistency.

| Term or idea | Meaning, use and example |
|---|---|
| **Flat-file system** | Stores data in a single table or file. Simple for small tasks, but repeated information can lead to update inconsistencies. |
| **Entity** | A type of thing to record, such as Student, Course or Enrolment. |
| **Attribute / field** | A property of an entity, represented by a column, such as StudentName. |
| **Record** | One row, representing an occurrence of the entity. |
| **One-to-one** | One row relates to at most one row on the other side, subject to the chosen optionality rules. |
| **One-to-many** | One parent may relate to many child rows. For example, one department has many courses, while each course belongs to one department. |
| **Many-to-many** | Each side may relate to many rows on the other side. Resolve this using a linking table, for example Enrolment between Student and Course. |
| **ER diagram** | Shows entity types, their keys and the cardinality of relationships. Read both ends and distinguish “may” from “must”. |

## 7.2 Keys and integrity

| Term or idea | Meaning, use and example |
|---|---|
| **Super key** | Any set of attributes that uniquely identifies a row, even if it includes unnecessary attributes. |
| **Candidate key** | A minimal super key: removing any part would lose uniqueness. A table may have several candidate keys. |
| **Primary key** | The chosen candidate key. It uniquely identifies each row and cannot be null. |
| **Composite key** | A key made of more than one attribute, such as (StudentID, CourseID) in Enrolment. |

| Term or idea | Meaning, use and example |
|---|---|
| **Foreign key** | An attribute or group of attributes referring to a candidate key, usually the primary key, in another table or the same table. It links related rows. |
| **Entity integrity** | Each row has a unique, non-null primary key. |
| **Referential integrity** | A non-null foreign-key value must match an existing referenced key. Prevents a loan from referring to a non-existent student, for example. |
| **Data dictionary** | Documents each field's name, meaning, data type, size where relevant, constraints and key relationships. |

| Term or idea | Meaning, use and example |
|---|---|
| **Data types** | Choose text for names and identifiers with leading zeros, numbers for quantities, date/time for dates and Boolean values for true/false attributes. Implementations vary by DBMS. |
| **NOT NULL** | Requires a value rather than the special missing/unknown marker NULL. |
| **UNIQUE** | Prevents duplicate values or combinations according to the DBMS's null-handling rules. |
| **DEFAULT** | Supplies a value when an insert omits that field. It does not validate every later change. |
| **CHECK** | Requires a condition, such as Score between 0 and 100. Combine with NOT NULL when a value is mandatory. |
| **Validation rules** | Presence, range, length and format checks reject unsuitable entries. A correctly formatted email address is not necessarily a real or correct address. |

### 7.3 Normalisation to 3NF

Normalisation organises related facts to reduce redundancy and **update**, **insertion** and **deletion anomalies**. It is not simply splitting a table into smaller pieces at random.

| Term or idea | Meaning, use and example |
|---|---|
| **Functional dependency** | X determines Y if each X value identifies exactly one Y value in the relation. It reflects the meaning of the data, not an accidental pattern in a few sample rows. |
| **1NF** | Each cell contains one value of its intended type, with no repeating groups. Rows must be identifiable. A list of several course IDs in one cell violates the intended relational structure. |
| **2NF** | In 1NF, and each non-key attribute depends on the whole of every candidate key, not just part of a composite candidate key. Removes partial dependencies. |
| **3NF** | In 2NF, with no inappropriate transitive dependencies of non-key attributes on keys. At this level, store each non-key fact with the entity it describes rather than through another non-key fact. |

**Worked example: Normalising enrolments**

Suppose a 1NF table has columns StudentID, CourseID, StudentName, CourseName, TutorID, TutorName and Mark. Its key is (StudentID, CourseID). Assume each student has one recorded name, each course has one tutor and each tutor has one recorded name.

1. StudentName depends only on StudentID, and course details depend only on CourseID. These partial dependencies violate 2NF.
2. Create Student(StudentID, StudentName), Course(CourseID, CourseName, TutorID, TutorName) and Enrolment(StudentID, CourseID, Mark).
3. In Course, CourseID determines TutorID, and TutorID determines TutorName. This transitive dependency motivates Tutor(TutorID, TutorName).
4. The resulting Course table is Course(CourseID, CourseName, TutorID). Enrolment's two IDs are foreign keys as well as its composite primary key.

Now a tutor's name is updated once. Tutor and course details can exist before any student enrols, and deleting the final enrolment need not delete the course. More tables can require more joins, so normalisation is primarily about sound structure and integrity, not a guarantee of faster queries.

## 7.4 SQL queries

The following examples use the Student, Course and Enrolment design above. SQL syntax can vary slightly by DBMS. Text literals use single quotes. A SELECT query does not require a particular row order unless ORDER BY specifies one.

```
SELECT StudentID, StudentName
FROM Student
WHERE StudentName = 'Amina'
ORDER BY StudentName ASC;

SELECT StudentID, CourseID, Mark,
       Mark + 5 AS AdjustedMark
FROM Enrolment
WHERE Mark >= 50 AND Mark <= 100
ORDER BY Mark DESC;
```

| Term or idea | Meaning, use and example |
|---|---|
| **SELECT** | Chooses fields or expressions to return. `SELECT *` returns all columns, but named columns make the intended output clearer. |
| **FROM** | Identifies the source table or tables. |
| **WHERE** | Filters rows using comparisons and logical operators. Use parentheses for mixed AND and OR conditions. |
| **ORDER BY** | Sorts results. ASC means ascending and DESC descending. |
| **Calculated field** | An expression in the result, for example `Mark + 5 AS AdjustedMark`. It does not automatically modify the stored Mark. |
| **INNER JOIN** | Combines matching rows from tables using a specified relationship. Missing matches are excluded. |
| **Subquery** | A query nested inside another query, used for tasks such as filtering against a calculated average or a set of IDs. |

**Worked example: Joining related records**

```
SELECT s.StudentName, c.CourseName, e.Mark
FROM Student AS s
INNER JOIN Enrolment AS e ON s.StudentID = e.StudentID
INNER JOIN Course AS c ON e.CourseID = c.CourseID
WHERE e.Mark >= 50
ORDER BY s.StudentName, c.CourseName;
```

The ON conditions match keys. Joining on an unrelated field can give incorrect combinations even if the query executes successfully.

**Worked example: A subquery**

```
SELECT StudentID, CourseID, Mark
FROM Enrolment
WHERE Mark > (SELECT AVG(Mark) FROM Enrolment);
```

The subquery produces the overall average mark. The outer query returns enrolments above it. Aggregate functions such as AVG, SUM, MIN and MAX normally ignore NULL values. Use `IS NULL`, not `= NULL`, to test for a missing value.

## 7.5 Creating and changing data

```
CREATE TABLE Student (
    StudentID INTEGER PRIMARY KEY,
    StudentName VARCHAR(80) NOT NULL
);

INSERT INTO Student (StudentID, StudentName)
VALUES (101, 'Amina');

UPDATE Enrolment
SET Mark = 78
WHERE StudentID = 101 AND CourseID = 20;

DELETE FROM Enrolment
WHERE StudentID = 101 AND CourseID = 20;
```

**Remember.** UPDATE or DELETE without an appropriate WHERE condition can affect every row. A change must also satisfy key and validation constraints.

## 7.6 Forms, reports, testing and performance

| Term or idea | Meaning, use and example |
|---|---|
| **Form** | An interface for entering and browsing records. Use clear labels, sensible ordering, accessible contrast and helpful validation messages. |
| **Controls** | Text boxes accept free text; combo boxes restrict choices; check boxes suit Boolean values; buttons perform actions such as Save, Next or Exit. |
| **Report** | A formatted presentation of data for reading or printing. It may use a query, calculated values, grouping, sorting and totals. Label units and headings clearly. |

| Term or idea | Meaning, use and example |
|---|---|
| **Database testing** | Test valid and invalid inserts, duplicate keys, missing foreign-key targets, query results, forms and reports. Use small data sets with known expected results. |
| **Index** | An additional structure that helps find or order rows. It can accelerate searches but consumes storage and adds maintenance work when records change. |
| **Evaluation** | Check correctness, usability, speed and compliance with the original requirements. Assess how record growth and simultaneous users may affect the design. |

**Worked example: A database test plan entry**

Test: insert an enrolment for StudentID 999 when no such student exists. Expected result: the foreign-key constraint rejects it. Actual result: record what the DBMS does. A successful query execution alone does not prove the data is correct.

**Check your understanding.** Why should StudentName not usually be the primary key? What distinguishes a partial dependency from a transitive dependency?

# 8 Networks and communication (LO8)

## 8.1 Network types, models and connections

A network connects devices so they can exchange data and share resources. The internet is a worldwide interconnection of networks. The World Wide Web is one service operating over it.

| Term or idea | Meaning, use and example |
|---|---|
| **PAN** | A personal area network connects nearby personal devices, such as a phone and wireless headset. |
| **LAN** | A local area network covers a limited site, such as a school or office, usually under one organisation's management. |
| **WAN** | A wide area network spans larger geographical areas and often uses a telecommunications provider's infrastructure. |
| **Client–server** | Clients request services from dedicated servers. Central management and backup are easier, but servers require administration and service availability must be planned. |
| **Peer-to-peer** | Devices can both request and provide resources. Simple for some small networks, but consistent access control, backup and administration become harder as the network grows. |
| **Wired connection** | Copper or fibre links can offer predictable performance and reduced radio interference. Installation costs and physical connections limit mobility. |
| **Wireless connection** | Radio links provide mobility and convenient access. Range, obstacles, interference, shared capacity and authentication affect reliability and security. |

## 8.2 Topologies

Topology describes how nodes are connected, physically or logically.

| Term or idea | Meaning, use and example |
|---|---|
| **Star** | Every end device connects to a central device, usually a switch. One end-device link failure normally affects only that device. Failure of the central switch can affect the whole star. More cabling is needed than in some shared-medium designs. |
| **Ring** | Nodes connect in a cycle. Access can be orderly, for example with token passing. A break can disrupt a simple ring unless redundancy or bypass mechanisms exist. Adding nodes may require reconfiguration. |
| **Mesh** | Nodes have several interconnections. Alternative routes improve resilience, but additional links cost more and complicate management. In a full undirected mesh of $n$ nodes there are $n(n-1)/2$ links. |

**Worked example: Choosing a topology**

A classroom LAN often suits a star: individual faults are easy to isolate and devices can be added to a switch. A critical inter-site service may justify redundant routes or a partial mesh to avoid a single link failure disconnecting a site.

## 8.3 Networking hardware

| Term or idea | Meaning, use and example |
|---|---|
| **NIC** | A network interface card or integrated adapter connects a device to a network. It may support Ethernet or wireless communication. |
| **Hub** | Repeats incoming signals to other ports. Devices share the medium, which can cause collisions in classic half-duplex Ethernet. Hubs do not selectively forward by destination MAC address. |
| **Switch** | Learns which MAC addresses are reachable on its ports and forwards frames accordingly. Full-duplex switched links avoid the classic shared-medium collision problem. Unknown destinations may be flooded. |
| **Router** | Forwards packets between IP networks using routing information. It selects a next hop towards the destination. |
| **Modem** | Converts signals to and from a form suitable for a particular access link. A home gateway may combine modem, router, switch and access-point functions. |
| **Wireless access point** | Connects wireless clients to a network. It is not necessarily a router, although consumer devices often combine both roles. |

## 8.4 Packets, addresses and performance

| Term or idea | Meaning, use and example |
|---|---|
| **Packet** | A unit of network-layer data with addressing and other control information. A message may be split across packets, which can take different routes or arrive out of order. |
| **Frame** | A link-layer unit used on a local connection, such as Ethernet. Frames carry packets between adjacent network nodes. |
| **MAC address** | A link-layer identifier used for local frame delivery. MAC addresses can be changed or randomised and should not be treated as proof of a person's identity. |
| **IP address** | A logical address used for delivery across IP networks. IPv4 uses 32 bits; IPv6 uses 128 bits. An address can change as a device changes networks. |
| **Bandwidth** | The capacity of a link, typically bits per second. |
| **Throughput** | The actual rate of successful data transfer. Protocol overhead, congestion and device limits can make it lower than link bandwidth. |
| **Latency** | The delay in communication, such as the time for a request and response. A high-bandwidth connection can still have high latency. |
| **QoS** | Quality of Service prioritises selected traffic, such as delay-sensitive voice. It manages competing traffic but does not create unlimited capacity. |

**Worked example: Transfer-time estimate**

An 80 MB file contains $80 \times 8 = 640$ megabits using decimal units. Over a link delivering 20 megabits/s, the idealised transfer time is $640/20 = 32$ seconds. Real transfers can take longer because of overhead and changing throughput. Do not confuse MB/s with Mb/s.

## 8.5 Protocols and layers

A protocol is an agreed set of communication rules, including message formats and expected behaviour. Layering separates responsibilities and allows parts of a network system to change without redesigning every application.

| TCP/IP layer | Main responsibility | Examples |
|---|---|---|
| Application | Services and application message formats | HTTP, DNS, FTP |
| Transport | Communication between application endpoints | TCP, UDP |
| Internet | Logical addressing and packet routing | IP |
| Link | Transfer over the local network medium | Ethernet, Wi-Fi |

| Term or idea | Meaning, use and example |
|---|---|
| **IP** | Provides addressing and best-effort packet delivery. It does not itself guarantee delivery or ordering. |
| **TCP** | Provides an ordered byte stream with mechanisms for reliability, retransmission, flow control and congestion control. Useful when missing or reordered data must be handled before the application receives the stream. |

| Term or idea | Meaning, use and example |
|---|---|
| **UDP** | Sends datagrams without TCP's built-in delivery and ordering guarantees. Applications can use it when timing or simple message exchange matters, and may add their own reliability. |
| **HTTP** | A request–response application protocol used for web resources. Plain HTTP does not encrypt the traffic. |
| **HTTPS** | HTTP protected with TLS. Helps provide confidentiality, integrity and server authentication when certificates are properly validated. It does not guarantee the site's content is honest. |
| **TLS / SSL** | TLS is the modern secure-channel protocol. SSL is its obsolete predecessor, though the old name is sometimes used informally. TLS combines cryptographic mechanisms rather than using only one kind of encryption. |
| **FTP** | Transfers files. Traditional FTP does not protect credentials and contents with encryption. FTPS uses TLS; SFTP is a different protocol operating over SSH. |
| **DNS** | Resolves names such as a website hostname into information including IP addresses. A domain name and an IP address are different identifiers. |
| **Wi-Fi (802.11)** | A family of wireless local-network standards. Actual performance depends on signal conditions, configuration and shared radio use. Use appropriate wireless security rather than assuming radio access is private. |
| **Bluetooth** | A family of short-range wireless technologies commonly used for personal devices and peripherals. Pairing and access controls still matter. |

### 8.6 Securing a network

Combine authentication, appropriate permissions, encryption, filtering, updates, monitoring and tested backups. These address different risks and should reinforce one another. Authentication checks who a user is; authorisation decides what that user is allowed to do. A trusted user can still make mistakes or misuse access.

**Check your understanding.** Why can a video call lag even when a speed test shows high bandwidth? How does a switch differ from a router?

## 9 Cyber security, legislation, ethics and AI (LO9)

### 9.1 Security goals and risks

| Term or idea | Meaning, use and example |
|---|---|
| **Confidentiality** | Only authorised parties can access information. A stolen customer list is a confidentiality failure. |
| **Integrity** | Information remains accurate and protected from unauthorised or accidental alteration. An altered payment amount is an integrity failure. |
| **Availability** | Authorised users can access systems and data when needed. A service outage is an availability failure. |

| Term or idea | Meaning, use and example |
|---|---|
| **Threat** | A potential cause of harm, such as an attacker, malicious software or a damaging event. |
| **Vulnerability** | A weakness a threat could exploit, such as unpatched software or excessive access permissions. |
| **Risk** | The possibility and impact of harm in a particular situation. Prioritise controls using both likelihood and consequence. |

## 9.2 Threats and appropriate responses

| Threat | What it means | Relevant mitigation |
|---|---|---|
| Virus | Malicious code that infects a host file or program and spreads when it executes. | Malware scanning, patching and controlling untrusted files. |
| Worm | Malware capable of spreading between systems without attaching itself to a host file. | Patching, network segmentation and monitoring. |
| Trojan | Malicious software presented as something useful or harmless. | Trusted software sources, restricted privileges and user awareness. |
| Ransomware | Malware that may encrypt data, disrupt access and threaten publication of stolen information. | Patching, access controls, isolated or immutable backups and tested recovery. Backups do not undo a data leak. |
| Spyware | Secretly gathers information about activity or data. | Anti-malware, permission control and careful software installation. |
| Phishing | Deceptive messages try to steal information or persuade a user to perform an action. | Awareness, independent verification, filtering and appropriate MFA. |
| Spear phishing | Phishing targeted using information about a particular person or organisation. | Verify sensitive requests through a known independent channel. |
| Social engineering | Manipulates people into revealing information or bypassing procedures, including by phone or in person. | Clear procedures, training and verification of identity and authority. |
| DoS / DDoS | Denial of service exhausts or disrupts resources. Distributed attacks use multiple sources. | Rate limiting, upstream protection, filtering and resilient service design. |
| Man-in-the-middle | An attacker intercepts or interferes with communication between parties. | Authenticated encryption, valid certificate checks and secure network configuration. |
| Insider threat | An authorised person causes harm deliberately or accidentally. | Least privilege, logging, separation of duties and training. |

A single incident can damage all three CIA goals and cause financial loss, reputational harm and legal consequences. Controls reduce risk; no single product guarantees security.

## 9.3 Technical and organisational controls

| Term or idea | Meaning, use and example |
|---|---|
| **Firewall** | Allows or blocks traffic according to rules. It cannot determine that every allowed application message is harmless. |
| **Anti-malware** | Detects or blocks malicious activity using signatures and other analysis. New or concealed threats may evade detection. |
| **Intrusion detection** | Monitors for suspicious activity and raises alerts. An intrusion-prevention system may also block traffic. Alerts still require investigation. |
| **Passwords** | Use long, unique passwords and suitable password policies. Password managers help avoid reuse. A password alone can be stolen. |
| **MFA / 2FA** | Requires independent factor categories, such as something known, possessed or inherent. Two passwords are still one factor category. A one-time code can represent possession; biometrics represent an inherent characteristic. |
| **Least privilege** | Give users and programs only the access needed for their work. Review access when roles change. |
| **Updates and patches** | Repair known flaws. Maintain an inventory so unsupported or forgotten devices are not missed. |
| **Backups** | Keep separate recoverable copies. Choose frequency according to acceptable data loss, keep suitable off-site or isolated copies, and test restoration. |
| **Audit trail** | Records actions with useful identity and timing information. Logs support investigation and accountability but need protection and review. |
| **Incident response** | Plan who will identify, contain, investigate, recover from and communicate an incident. Preserve relevant evidence and learn from the cause. |
| **Security awareness** | Helps users recognise suspicious requests and report mistakes promptly. Processes should make secure behaviour practical. |

## 9.4 Encryption and related concepts

| Term or idea | Meaning, use and example |
|---|---|
| **Encryption** | Transforms plaintext into ciphertext using a key. Decryption recovers the plaintext with the appropriate key. Secure key management is essential. |
| **Symmetric encryption** | Uses a shared secret key. Efficient for bulk data, but the parties need a secure way to obtain and protect the shared secret. |
| **Asymmetric cryptography** | Uses related public and private keys. It supports operations such as encryption, digital signatures and key agreement, depending on the scheme. The private key must remain secret. |
| **Hashing** | Produces a fixed-length digest. It is designed to be one-way, unlike encryption. Password storage should use a suitable salted password-hashing method, not reversible encryption or a plain fast hash. |
| **Digital signature** | A private-key operation allows verification using a corresponding public key. Supports integrity and authenticity, but does not by itself conceal the message. |

**Worked example: WannaCry and the NHS**

The 2017 WannaCry attack disrupted NHS services. The National Audit Office's investigation illustrates why patch management, supported systems, coordinated preparation and recovery arrangements matter together. Treat this as a case about prevention and service availability, not evidence that buying antivirus alone is sufficient. See the source list for the investigation.

## 9.5 Legislation and jurisdiction

The syllabus names the following legislation. The three named UK Acts are UK examples; do not describe them as Irish Acts. GDPR is an EU regulation. Jurisdiction and the facts of a situation determine which rules apply. This section explains the course concepts rather than giving advice on a particular case.

| Term or idea | Meaning, use and example |
|---|---|
| **GDPR** | Regulates personal-data processing. Organisations need a lawful basis, appropriate safeguards and respect for applicable individual rights. Consent is one lawful basis, not the only one. |
| **Computer Misuse Act 1990 (UK)** | Addresses unauthorised access and other specified misuse of computer systems. Being technically able to access a system does not mean having permission. |
| **Freedom of Information Act 2000 (UK)** | Provides rights to request recorded information held by covered public authorities, subject to exemptions and other conditions. It is not a right to see all private organisations' records. |
| **Copyright, Designs and Patents Act 1988 (UK)** | Protects specified categories of creative work, including relevant software and digital content. Copying and reuse depend on rights, licences and applicable exceptions. Material being online does not make it free to reuse. |
| **Intellectual property** | Legal rights concerning creations such as software, content, inventions and brands. Copyright, patents, trademarks and database rights are different forms with different conditions. |
| **Data controller** | Determines the purposes and means of processing personal data. It is responsible for demonstrating compliance with its obligations. |
| **Data processor** | Processes personal data on behalf of, and under instructions from, a controller. It also has its own legal obligations. |

**GDPR principles in plain language:** process lawfully, fairly and transparently; use data for specified legitimate purposes; collect only what is needed; keep it accurate; retain identifiable data only as long as necessary; protect it; and demonstrate accountability. A retention policy states how long records are kept and when they are reviewed or deleted.

Individuals have rights such as access and correction, with other rights, including erasure, restriction, portability and objection, subject to their relevant conditions. Non-compliance can lead to regulatory action and other consequences. Check the applicable jurisdiction and current official guidance when applying these ideas.

## 9.6 Ethical responsibilities

Law asks what is legally permitted or required. Ethics asks what is responsible and fair. A lawful system can still have unacceptable consequences.

| Term or idea | Meaning, use and example |
|---|---|
| **Privacy** | People should understand and have appropriate control over how information about them is used. Avoid collecting data merely because it is technically available. |
| **Consent and choice** | Explain relevant choices clearly. A confusing interface or unnecessary pressure can undermine meaningful choice. |
| **Security versus privacy** | Monitoring can help detect attacks but may expose private activity. Consider necessity, proportionality, access limits and retention. |
| **Accessibility** | Design so users with different abilities can participate, for example through readable text, keyboard navigation and understandable error messages. |
| **Social impact** | Consider who benefits, who may be excluded, effects on work and skills, and environmental costs such as energy use and electronic waste. |
| **Professional responsibility** | Test systems, report limitations honestly, protect confidential information and correct known harmful errors. |

## 9.7 Artificial intelligence and critical use

| Term or idea | Meaning, use and example |
|---|---|
| **AI** | Systems designed to perform tasks associated with intelligent behaviour, such as classification, prediction or language generation. It is a broad category, not one algorithm. |
| **Machine learning** | A branch of AI in which models learn patterns from examples rather than relying only on explicitly written decision rules. |
| **Training and inference** | Training adjusts a model using data. Inference uses the trained model to produce a prediction or output for new input. |
| **Bias and fairness** | Unrepresentative data, labels, objectives or deployment conditions can create unfair outcomes. Evaluate relevant groups and consequences, not just one overall accuracy figure. |
| **Hallucinated output** | A plausible but unsupported or incorrect generated statement, source or calculation. Fluent wording does not establish truth. |
| **Deepfake** | Synthetic or manipulated media that can misrepresent a person's appearance, voice or actions. It can support impersonation and misinformation. |
| **AI in security** | Can help identify unusual patterns and prioritise alerts. False positives and false negatives remain possible, and attackers can also use AI for deceptive content or fraud. |
| **Human oversight** | People must be able to review important decisions, question evidence and correct errors. Responsibility does not disappear because software produced a result. |

**Worked example: Using an AI explanation responsibly**

Ask: "Explain binary search to a beginner. Use the sorted list [2,5,8,12,16]. Show low, high and middle for target 12." Then check the trace yourself, test any code and compare the explanation with trusted course material. Refine the prompt if the answer is unclear. Do not upload confidential records without authorisation.

For a system that selects applicants, ask whether the training data represents the applicants, whether the outcome is fair across relevant groups, whether reasons can be explained and whether someone can challenge an error. A high average accuracy does not answer all these questions.

**Check your understanding.** How do encryption and hashing differ? Why might an accurate-looking AI answer still need independent verification?

# 10 Revision checks and worked answers

## 10.1 Understanding assessment instructions

| Term or idea | Meaning, use and example |
|---|---|
| **State / identify** | Give the requested fact or name directly. |
| **Describe** | Say what something is like or outline what happens. |
| **Explain** | Give a reason or mechanism: how or why something happens. |
| **Compare** | Discuss similarities and differences using the same criteria. |
| **Analyse** | Break the problem into parts and explain how the parts interact. |
| **Evaluate / justify** | Weigh relevant benefits and limitations, then support a judgement for the given situation. |
| **Trace / calculate** | Show intermediate values and working. Give units where relevant. |

## 10.2 Answers to the section checks

1. **LO1:** Clock rate measures cycles, not useful work completed. Architecture, instructions per cycle, core count, cache, RAM, storage and the workload can affect performance. For example, more RAM may prevent slow paging.

2. **LO2:** $640 \times 480 \times 8/8 = 307\,200$ bytes. A PNG uses compression and has additional structural information, so its saved size need not equal the raw pixel count.

3. **LO3:** $X \vee (X \wedge Y) = X$ by absorption. Three independent inputs give $2 \times 2 \times 2 = 8$ combinations.

4. **LO4:** Undo suits a stack, print jobs a queue and lookup by student ID a dictionary. Binary search requires ordering to justify discarding half the data; with unsorted data it may discard the target.

5. **LO5:** Decomposition separates validation, summing and reporting. Abstraction retains the scores and rules needed for the mean while ignoring irrelevant details such as students' favourite colours.

6. **LO6:** Agile allows feedback to influence the next iteration. Frequent changes can complicate scope and planning, so priorities, acceptance criteria and documentation still need management.

7. **LO7:** Names can repeat or change. A stable unique ID is safer. A partial dependency is on part of a composite key; a transitive dependency reaches another non-key fact through an intermediate attribute.
8. **LO8:** High latency, jitter, congestion or packet loss can disrupt a call despite high capacity. A switch forwards frames within a link-layer network; a router forwards IP packets between networks.
9. **LO9:** Encryption is reversible with a suitable key; hashing is intended to be one-way. AI output may contain errors, fabricated sources or bias, even when it reads confidently.

### 10.3 Mixed practice

1. Convert $3A_{16}$ to binary and decimal.
2. Interpret $11110110_2$ as an eight-bit two's-complement integer.
3. Calculate raw size for 10 seconds of mono audio at 8000 samples/s and 8 bits/sample.
4. Simplify $\neg(A \wedge \neg B)$.
5. In a four-variable K-map, all cells in rows AB=11 and AB=10 are 1 and all others are 0. Simplify.
6. What does `list(range(1, 6, 2))` contain?
7. A function prints 8 but has no return statement. What does assigning its result to x store in x?
8. Give the worst-case time complexity of linear search, binary search and bubble sort. State the prerequisite for binary search.
9. Why can a database query run without errors and still be wrong?
10. A user has a strong password, but enters it on a phishing site. Give two measures that could reduce the risk or consequences.

### 10.4 Mixed-practice answers

1. $0011\,1010_2$, and $3 \times 16 + 10 = 58$.
2. $-128 + 64 + 32 + 16 + 4 + 2 = -10$.
3. $8000 \times 8 \times 1 \times 10/8 = 80\,000$ bytes before headers.
4. $\neg A \vee B$, using De Morgan and double negation.
5. A. A stays 1 while B, C and D all vary within the group of eight.

1,3,5 . The stop value 6 is excluded.

6. `None`. Printed output and returned values are separate.
7. $O(n)$, $O(\log n)$ and $O(n^2)$ respectively. Binary search needs sorted data and efficient access to the middle element for that usual bound.
8. A wrong filter or join can return the wrong records. Compare against known expected results and requirements.
9. Verify the site's identity before signing in, provide awareness training, use appropriate MFA and promptly revoke compromised access. The suitable combination depends on the scenario.

### 10.5 Coverage map

| Criteria | Where to revise in this reference |
|---|---|
| AC1.1–1.4 | Section 1: components, architectures, registers, buses, DFDs, memory, OS management and performance. |
| AC2.1–2.5 | Section 2: number systems, signed values, arithmetic, text, images, sound and compression. |
| AC3.1–3.3 | Section 3: problem analysis, logic, gates, truth tables, Boolean laws, K-maps, flowcharts and pseudocode. |
| AC4.1–4.4 | Section 4: Python constructs, functions, files, turtle, testing, optimisation, searching, sorting and data structures. |
| AC5.1–5.4 | Section 5: computational thinking, forms of abstraction, problem-solving approaches and solution evaluation. |
| AC6.1–6.3 | Section 6: software types, languages, translators, paradigms, development approaches and their trade-offs. |
| AC7.1–7.2 | Section 7: database design, keys, integrity, normalisation, SQL, forms, reports, testing and optimisation. |
| AC8.1–8.3 | Section 8: networks, topologies, devices, addressing, protocols, performance and security. |
| AC9.1–9.3 | Section 9: threats, controls, law, ethics and critical use of AI. |

## Sources and further reading

The course coverage is based on the supplied NCUK *IFY Computer Science Module Specification v1.2* (2026), particularly pages 7–26. The reference uses original explanatory summaries and examples. Practise alongside your lectures, exercises and the supplied exam papers.


1. NCUK. *IFY Computer Science Module Specification v1.2*, 2026. Learning outcomes 1–9 and assessment criteria.

2. NCUK. *IFY Computer Science Exemplar Teaching Plan v1.1.* Suggested teaching sequence and activities.

3. Heathcote, P. *AS and A Level OCR Computer Science.* PG Online, 2016. Course core text; Boolean topics are in chapters 40–42.

4. Wentworth, P., Elkner, J., Downey, A. J. and Meyers, C. *How to Think Like a Computer Scientist: Learning with Python 3.* Course core text. `https://openbookproject.net/thinkcs/python/english3e/`

5. Data Protection Commission. Principles of Data Protection. `https://www.dataprotection.ie/en/individuals/data-protection-basics/principles-data-protection`

6. Data Protection Commission. Your Rights under the GDPR. `https://www.dataprotection.ie/en/individuals/rights-individuals-under-general-data-protection-regulation`

7. Data Protection Commission. What is a Data Controller and a Data Processor? `https://www.dataprotection.ie/en/faqs/general/what-data-controller-and-data-processor`

8. UK legislation. Computer Misuse Act 1990. `https://www.legislation.gov.uk/ukpga/1990/18/contents`

9. UK legislation. Freedom of Information Act 2000. https://www.legislation.gov.uk/ukpga/2000/36/contents

10. UK legislation. Copyright, Designs and Patents Act 1988. https://www.legislation.gov.uk/ukpga/1988/48/contents

11. National Audit Office. *Investigation: WannaCry cyber attack and the NHS*, 2017. https://www.nao.org.uk/press-releases/investigation-wannacry-cyber-attack-and-the-nhs/

*Official legal and data-protection sources checked on 5 November 2026. Apply the jurisdiction and version specified in an assessment.*